LOOKMATE

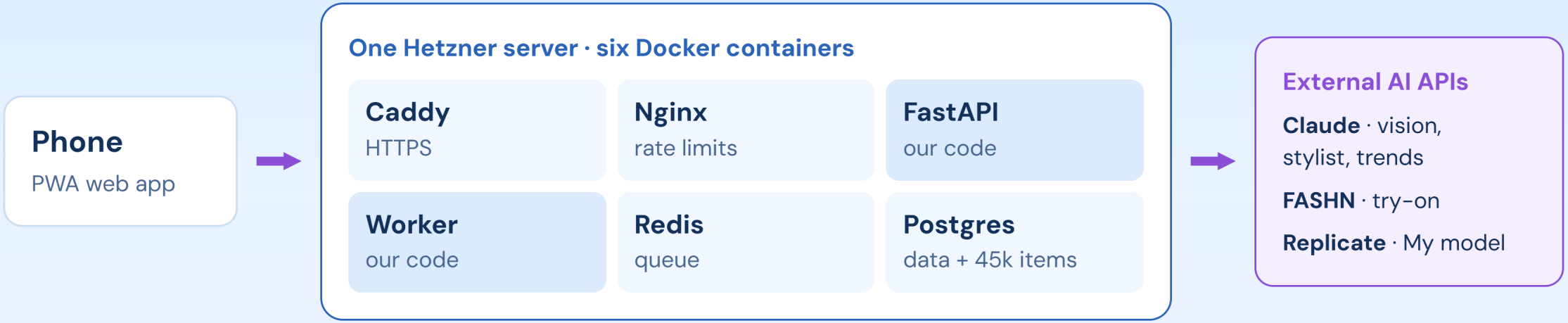
your style buddy

How it works

Pre-trained models for understanding, deterministic code for judging.

LOOKMATE

System architecture



On our CPU: only the small bge text-embedding model

CI: GitHub Actions runs every test on each push

LOOKMATE

How we chose the image models

Try-on

dress my model

IDM-VTON
Cold starts of minutes; non-commercial licence

→

Nano Banana
Fast, whole outfit; garment detail weaker

→

FASHN ✓ final
Try-on specialist, no cold start, commercial use

My model

draw me once

Nano Banana Pro ✓ final
A new full-body figure, not a garment swap; kept the face best

Fallbacks
Plain Nano Banana if Pro fails; “use my original photo”

Text + vision

read and search

Claude
Photo → schema, stylist, trend research

bge-small
Text search on CPU: free, no GPU, no API call

LOOKMATE

Where the 45,000 products come from

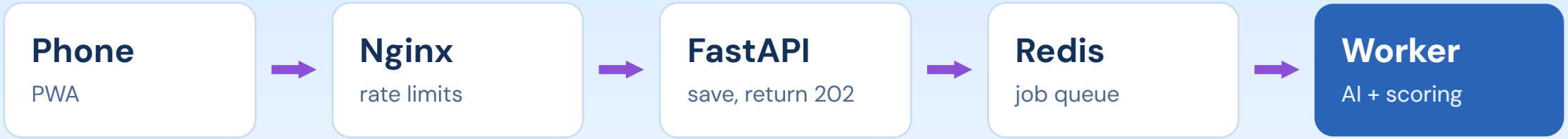
Dataset	Kept	Why / what we had to fix
Amazon Reviews 2023	40,000	Real prices and photos; filtered from ~7.2M listings
ASOS	2,500	Real prices and photos; one row per size, so deduplicated
Polyvore	2,500	Designer pieces; no prices, so a fixed designer-band price
H&M	dropped	First choice, but its image bucket was deleted

Import pipeline: stream → filter → deduplicate → embed in chunks → swap in one transaction

Demo scope: women's fashion only, to match the body-shape fit rules and my eval photos.

LOOKMATE

How a request flows



Why a queue

Model calls take 5 s to 1 min. The request never waits.

Reliable

At-least-once delivery, idempotent jobs, retry with backoff.

Runs anywhere

One Docker Compose stack, plus Postgres. Works without an API key.

LOOKMATE

One photo, four steps

Step	Who does it	What happens
1. Perceive	Claude vision	Photo → each piece in a Pydantic schema
2. Recall	bge-small embeddings	Cosine search over 45k products → top 500
3. Filter	Hard rules	Type, length, sleeve, season, leg, colour, shade, pattern
4. Score	Weighted formula	Score + one reason per term → 4 distinct picks
Lookbook	Claude stylist	Picks one outfit from the top 5 per slot

Purple = pre-trained model · dark blue = our deterministic, unit-tested code

LOOKMATE

Codebase map: follow one photo

#	File	What it does
1	api/analysis.py, api/looks.py	Receive photo, dedupe, queue the job, return 202
2	jobqueue.py	Redis queue that never loses a job
3	worker.py	Takes jobs, calls the AI, saves results
4	llm/schemas.py, llm/client.py	The schema Claude must fill, and the call
5	catalog/ (embedder, index)	bge vectors and the cosine search
6	services/ (match, ranking, dupes)	Rules, scoring, the Find dupes recipe
7	services/lookbook.py, stylist.py	Outfits and the Claude stylist

Folders: **api** = the doors · **llm** = talking to Claude · **catalog** = products + search · **services** = the logic · **tryon** = FASHN / Replicate

LOOKMATE

The schema is the contract

```
class DetectedItem(BaseModel):
category: "top" | "bottom" | …
name: str
colour: str
fit: str
sleeve: "sleeveless" | "short" | "long"
warmth: "summer" | "all-season" | "winter"
partial: bool
```

1. Tells Claude what to return

Field descriptions are part of the prompt.

2. Validates the answer

Wrong shape → job fails, bad data never flows on.

3. Hands typed objects on

Code reads item.sleeve, never raw model text.

LOOKMATE

How a pick is scored

0.60
Similarity
cosine, from recall

0.15
Style
tags + your memory

0.15
Price
cheaper is better

0.15
Colour
same family + shade

0.10
Body fit
your shape

```
score = 0.60·sim + 0.15·style + 0.15·price + 0.15·colour + 0.10·fit
```

Similarity dominates; the other terms only reorder close candidates. Each term also writes a reason on the card.

LOOKMATE

Worked example: $30 light blue jeans

Term	How it's worked out	Points
Similarity	cosine 0.82 × 0.60	0.492
Style	"basic" = minimalist, 0.5 × 0.15	0.075
Price	1 − 30 ÷ 100 = 0.7, × 0.15	0.105
Colour	same colour and shade, 1 × 0.15	0.150
Body fit	no fit rule applies	0
Total		0.822

Shown on the card

Same colour

Matches your minimalist style

Within your budget

LOOKMATE

Lookbook: same pipeline, its own score

	Find dupes	Lookbook
Query from	Claude's read of the outfit photo	Your palette + a style, per slot
Rules	Same type, colour, length, shade…	No avoided colour, one bright piece
Similarity	0.60	0.55
Palette	–	0.15
Style / price / fit	0.15 / 0.15 / 0.10	0.10 / 0.10 / 0.10
Colour match	0.15	– (palette does that job)
Last step	Top 4 distinct picks	Claude stylist picks 1 outfit from top 5

LOOKMATE

AI output is never trusted raw

Perception behind a schema

Claude's answer must match the Pydantic types or the job fails.

Stylist picks, never invents

Only from the top 5 scored candidates per slot; an unknown id is ignored.

Deterministic fallback

No key, an error or the daily cap: the scorer's picks stand.

Precision over fill

No match? An honest "not in our catalog" note, never a near-miss.

LOOKMATE

Testing and evaluation

267

unit + integration tests, offline

34

Playwright tests, one per bug I found

CI

boots the full Compose stack and runs a smoke test

Eval on my labelled photos	5k catalog	45k catalog
Find dupes precision@5 (24 photos)	88%	91%
Lookbook stylist approval (25 photos)	93%	75%
Avoided colours / outside price range	0	0